Loyola University Chicago — Department of Computer Science

COMP 150-003 Midterm Practice Problems

Fall 2026 · Instructor: Allan Miller

How to use this set. These problems have the **same format and point values** as the real midterm, but they are *not* the exam questions. Do them on paper, timed (90 minutes), with only your one handwritten sheet of notes. Then check yourself against the solutions, and compare your *reasoning*, not just your answers. Anything you got wrong goes on your notes sheet.

Question	Points	Score
True or False	15	
What Does It Do?	20	
Digital Information	10	
Names and Expressions	10	
Trace It	10	
Dictionaries	15	
Testing and Debugging	10	
Loops and Turtle	10	
Extra Credit: Classes (optional)	0	
Total:	100	

Problem 1: True or False (15 points)

Write **T** or **F** in each blank. (1 point each)

(a) (1 point) _____ `!=` means "not equal to."

(b) (1 point) _____ `range(1, 5)` includes the number 5.

(c) (1 point) _____ A list can contain the same value more than once.

(d) (1 point) _____ You can change one character of a string in place, e.g. `word[0] = "J"`.

(e) (1 point) _____ `10 // 3 == 3`

(f) (1 point) _____ `len([]) == 0`

(g) (1 point) _____ `bool("")`

(h) (1 point) _____ `"a" in "banana"`

(i) (1 point) _____ `2 ** 3 == 6`

(j) (1 point) _____ `int("7") + 1 == 8`

(k) (1 point) _____ A `for` loop over a dictionary visits its keys.

(l) (1 point) _____ An `elif` can appear without an `if` before it.

(m) (1 point) _____ `x += 1` does the same thing as `x = x + 1`.

(n) (1 point) _____ Comments change what a program does when it runs.

(o) (1 point) _____ In TDD, you refactor only while all your tests stay green.

Problem 2: What Does It Do? (20 points)

Circle the correct answer. (2.5 points each)

(a) (2½ points) What is printed?

```
a = "3"
b = "4"
print(a + b)
```

A. `7` B. `34` C. `3 4` D. an error

(b) (2½ points) What is `count` at the end?

```
count = 0
for i in range(2, 12, 3):
    count += 1
```

A. 3 B. 4 C. 10 D. 12

(c) (2½ points) Which word is printed the most times?

```
for n in range(1, 16):
    if n % 2 == 0:
        print("even")
    elif n % 3 == 0:
        print("three")
    else:
        print("other")
```

A. `even` B. `three` C. `other` D. They tie

(d) (2½ points) What is `len(set("banana"))`? A. 6 B. 4 C. 3 D. 2

(e) (2½ points) What is the value of `part`?

```
school = "Loyola Chicago"
part = school[0:6]
```

A. `"Loyola"` B. `"Loyol"` C. `"Loyola "` D. `"Chicago"`

(f) (2½ points) Which expression gives the *last* item of the list?

```
nums = [4, 8, 15, 16, 23, 42]
```

A. `nums[6]` B. `nums[-1]` C. `nums[len(nums)]` D. `nums[::-1]`

(g) (2½ points) Which expression tells you how many cities have a price?

```
prices = {"Chicago": 100, "Denver": 250, "Boston": 180}
```

A. `prices.count()`

B. `len(prices)`

C. `sum(prices.values())`

D. `len(prices["Chicago"])`

(h) (2½ points) Which line causes an error?

```
word = "abc".upper()      # line A
pair = (1, 2)             # line B
ages = {"Ana": 19}
print(ages.get("Bo"))     # line C
nums = [1, 2, 3]
print(nums[3])            # line D
```

A. A B. B C. C D. D

Problem 3: Digital Information (10 points)

(a) (4 points) Encode the string `Ok?` in ASCII. Give each character's decimal code and 8-bit binary code, then say how many bytes the string takes. (Hint: `A`=65, `a`=97, `?`=63.)

(b) (6 points) Convert the decimal number **58** to binary, octal, and hexadecimal. Show your work.

Problem 4: Names and Expressions (10 points)

(a) (4 points) Circle each *valid* Python variable name. For each invalid one, say why in a few words.

`my_var` `3dogs` `first name` `_count2` `for` `totalScore`

(b) (6 points) Evaluate each expression. (1 point each)

`23 // 5`		`-9 // 4`	
`23 % 5`		`2 + 3 * 4 ** 2`	
`9 / 3`		`not (True and False) or False`	

Problem 5: Trace It (10 points)

This loop adds up daily temperatures and counts the warm days.

```
temps = [3, 7, 2, 9, 4]
total = 0
warm_days = 0
for t in temps:
    total += t
    if t > 4:
        warm_days += 1
print(total, warm_days)
```

(a) (8 points) Fill in the trace table: one row per pass through the loop, showing the values *after* that pass.

pass	t	total	warm_days
1			
2			
3			
4			
5			

(b) (2 points) What is printed? _____

Problem 6: Dictionaries (15 points)

A budget app records each purchase as a `(category, amount)` pair:

```
purchases = [("food", 12), ("books", 30), ("food", 18), ("games", 5)]
```

(a) (8 points) Write `category_totals(purchases)` that returns a dictionary mapping each category to the total spent on it. For the list above it returns `{"food": 30, "books": 30, "games": 5}`.

(b) (7 points) Write `biggest_category(totals)` that returns the category with the most spending. Break ties **alphabetically** (for the example, `"books"` beats `"food"`). Return `None` for an empty dictionary. Use a loop, not `max(..., key=...)`.

Problem 7: Testing and Debugging (10 points)

This function should count how many words are *longer than* `n` letters. It has a bug.

```
def count_long_words(words, n):
    for w in words:
        count = 0
        if len(w) > n:
            count += 1
    return count
```

(a) (3 points) Trace `count_long_words(["hello", "world", "hi"], 3)`. What does it return, and what *should* it return?

(b) (3 points) Fix the bug. (You may mark up the code above or rewrite the lines.)

(c) (4 points) Write four `assert` tests for the fixed function: one *normal*, one *special*, and two *edge* cases. Add a short why-comment to each.

Problem 8: Loops and Turtle (10 points)

(a) (2 points) Write a `for` loop that builds the list `[2, 4, 6, ..., 20]`.

(b) (2 points) Write a `while` loop that builds the same list.

(c) (2 points) Given a list `nums`, write a list comprehension that keeps only the odd numbers.

(d) (4 points) Assume `import turtle` and `t = turtle.Turtle()`. The turtle starts at $(0, 0)$ facing right. Draw an equilateral triangle with sides of 100 whose **lower-left** corner is at $(0, 0)$.

Problem 9: Extra Credit: Classes (optional) (up to 10 extra-credit points)

```
class Pet:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(self.name + " makes a sound.")

class Dog(Pet):
    def speak(self):
        print(self.name + " says woof!")
```

(a) (3 bonus points) What does this print? `for p in [Pet("Nemo"), Dog("Rex")]: p.speak()`

(b) (4 bonus points) Rewrite `Pet` so its constructor also takes `age` and stores it. Add a method `birthday` that adds 1 to the age and prints the new age.

(c) (3 bonus points) Make a 2-year-old pet `"Nemo"` and a 5-year-old pet `"Tux"`, put them in a list, and call `birthday` on each.